

Heaps & Priority Queue Problems and Solution

1. Find Median from Data Stream

Approach 1: Two Heaps

Use two heaps:

- A max-heap to store the smaller half of the numbers.
- A min-heap to store the larger half.
- Balance the heaps to ensure their sizes differ by at most one.

Median:

- If heaps are of equal size, the median is the average of the roots.
- Otherwise, it's the root of the heap with more elements.

Pseudocode:

```
void findMedian(vector &nums) {  
    priority_queue maxHeap; //Left side  
    priority_queue, greater> minHeap; //Right side  
    for (int num : nums) {  
        if (maxHeap.empty() or num <= maxHeap.top()) {  
            maxHeap.push(num);  
        } else {  
            minHeap.push(num);  
        }  
    }  
    // Balance the
```

heaps

```
if (maxHeap.size() > minHeap.size() + 1) {  
    minHeap.push(maxHeap.top());  
    maxHeap.pop();  
} else if (minHeap.size() > maxHeap.size()) {  
    maxHeap.push(minHeap.top());  
    minHeap.pop();  
}  
  
}  
  
if (maxHeap.size() > minHeap.size()) {  
    cout << maxHeap.top();  
} else {  
    cout << (maxHeap.top() + minHeap.top()) / 2.0;  
}  
}
```

Dry Run:

Input: [5, 15, 1, 3]

Step 1: Insert 5 into maxHeap. Median = 5.

Step 2: Insert 15 into minHeap. Median = $(5 + 15) / 2 = 10$.

Step 3: Insert 1 into maxHeap (rebalance). Median = 5.

Step 4: Insert 3 into maxHeap (rebalance). Median = $(3 + 5) / 2 = 4$.

Output: Median = 4

Time Complexity: $O(\log N)$ per insertion, $O(1)$ for median

retrieval.

Space Complexity: $O(N)$ for heaps.

2. K Closest Points to Origin

Approach 1: Max-Heap

- Use a max-heap of size K to store points based on their distance from the origin.
- If the size exceeds K , remove the farthest point.

Pseudocode:

```
void kClosestPoints(vector< Point > &points, int K) {
    priority_queue< Point, vector< Point >, greater< Point > > maxHeap;
    for (auto point : points) {
        int dist = point.first * point.first + point.second * point.second;
        maxHeap.push({dist, {point.first, point.second}});
        if (maxHeap.size() > K) {
            maxHeap.pop();
        }
    }
    while (!maxHeap.empty()) {
        cout << maxHeap.top().second.first << " " << maxHeap.top().second.second
        <<
```

```
endl;  
maxHeap.pop();  
}  
}
```

Dry Run:

Input: [(1, 3), (-2, 2)], K = 1

Insert (1, 3) → Distance = 10.

Insert (-2, 2) → Distance = 8. Remove (1, 3). Output: (-2, 2)

- Time Complexity: $O(N \log K)$.
- Space Complexity: $O(K)$.

3. Last Stone Weight

Approach 1: Max-Heap

- Push all stone weights into a max-heap.
- Pop two largest stones, calculate the difference, and push the result back.
- Repeat until one or no stones remain.

Pseudocode:

```
void lastStoneWeight(vector &stones) {  
    priority_queue maxHeap(stones.begin(), stones.end());  
    while (maxHeap.size() > 1) {  
        int stone1 = maxHeap.top(); maxHeap.pop();  
        int stone2 = maxHeap.top();
```



```

maxHeap.pop();
if (stone1 != stone2) {
    maxHeap.push(stone1 - stone2);
}
}
cout << (maxHeap.empty() ? maxHeap.top());
}

```

Dry Run:

Input: [2, 7, 4, 1, 8, 1]

Pop 8, 7 → Push 1.

Pop 4, 2 → Push 2.

Pop 2, 1 → Push 1.

Pop 1, 1 → Nothing left.

Output: 0

- Time Complexity: $O(N \log N)$.
- Space Complexity: $O(N)$.

4. Kth Largest Element in an Array

Approach 1: Min-Heap

- Use a min-heap of size K .
- Insert elements and maintain the heap size to K by removing the

smallest.

Pseudocode:

```
void kthLargest(vector &nums, int K) {  
    priority_queue, greater> minHeap;  
    for (int num : nums) {  
        minHeap.push(num);  
        if (minHeap.size() > K) {  
            minHeap.pop();  
        }  
    }  
    cout << minHeap.top();  
}
```

Dry Run:

Input: [3, 2, 1, 5, 6, 4], K = 2

Insert 3 → [3]

Insert 2 → [2, 3]

Insert 1 → [2, 3]

Insert 5 → [3, 5]

Insert 6 → [5, 6]

Insert 4 → [5, 6]

Output: 5

Time Complexity: $O(N \log K)$

K).

Space Complexity: $O(K)$.

5. Sort Characters by Frequency

Approach 1: Frequency Map and Max-Heap

- Create a frequency map of characters.
- Use a max-heap to store characters by frequency.

Pseudocode:

```
void frequencySort(string s) {  
    unordered_map freq;  
    for (char c : s) freq[c]++;  
    priority_queue< maxHeap;  
    for (auto it : freq) {  
        maxHeap.push({it.second, it.first});  
    }  
    string result = "";  
    while (!maxHeap.empty()) {  
        result += string(maxHeap.top().first, maxHeap.top().second);  
        maxHeap.pop();  
    }  
    cout << result;  
}
```

Dry

Run:

Input: "tree"

Frequency map: {t: 1, r: 1, e: 2}.

Max-Heap: [(2, e), (1, t), (1, r)]. Output: "eetr" or "eert"

Time Complexity: $O(N \log N)$.

Space Complexity: $O(N)$.

6. Task Scheduler

Approach 1: Frequency Counting and Sorting

- Count the frequency of each task.
- Sort the tasks based on frequency.
- Calculate the idle slots required and fill them with remaining tasks.

Pseudocode:

```
void taskScheduler(vector &tasks, int n) {  
    vector freq(26, 0);  
    for (char task : tasks) {  
        freq[task - 'A']++;  
    }  
    sort(freq.rbegin(), freq.rend());  
    int maxFreq = freq[0];  
    int idleTime = (maxFreq - 1) * n;  
    for (int i = 1; i < 26; i++)
```

```

{
idleTime -= min(freq[i], maxFreq - 1);
}
idleTime = max(0, idleTime);
cout << tasks.size() + idleTime;
}

```

Dry Run:

Input: ['A', 'A', 'A', 'B', 'B', 'B'], n = 2

Frequency: [3, 3, 0...].

*Idle Time: $(3 - 1) * 2 = 4$.*

Fill idle slots: Remaining idle = 0. Output: 6

Time Complexity: $O(N + 26 \log 26) \approx O(N)$.

Space Complexity: $O(1)$.

7. Top K Frequent Elements

Approach 1: Frequency Map and Min-Heap

Count the frequency of elements.

Use a min-heap of size K to store elements based on frequency.

Pseudocode:

```

void topKFrequent(vector &nums, int K) {
unordered_map freq;
for (int num : nums)

```

```

{
freq[num]++;
}

priority_queue<vector<, greater>> minHeap;
for (auto it : freq) {
minHeap.push({it.second, it.first});
if (minHeap.size() > K) {
minHeap.pop();
}
}

while (!minHeap.empty()) {
cout << minHeap.top().second << " ";
minHeap.pop();
}
}

```

Dry Run:

Input: [1, 1, 1, 2, 2, 3], K = 2

Frequency: {1: 3, 2: 2, 3: 1}.

Min-Heap: [(2, 2), (3, 1)]. *Output:* 1, 2

Time Complexity: $O(N \log K)$.

Space Complexity: $O(N)$.

8. Merge

Intervals:

Approach 1: Sorting and Merging

- Sort intervals based on the starting time.
- Iterate through intervals and merge overlapping ones.

Pseudocode:

```
void mergeIntervals(vector<int> &intervals) {  
    sort(intervals.begin(), intervals.end());  
    vector<int> result;  
    for (auto interval : intervals) {  
        if (result.empty() or result.back().second < interval.first) {  
            result.push_back(interval);  
        } else {  
            result.back().second = max(result.back().second, interval.second);  
        }  
    }  
}
```

```
for (auto res : result) {  
    cout << "[" << res.first << ", " << res.second << "]" << " ";  
}  
}
```

Dry Run:

Input: [(1, 3), (2, 6), (8, 10), (15,

18)]

Sorted: [(1, 3), (2, 6), (8, 10), (15, 18)].

Merge (1, 3) and (2, 6) → [(1, 6), (8, 10), (15, 18)]. Output: [(1, 6), (8, 10), (15, 18)]

Time Complexity: $O(N \log N)$.

Space Complexity: $O(N)$.

9. Longest Substring Without Repeating Characters

Approach 1: Sliding Window

- Use two pointers and a set to maintain unique characters in the current window.
- Expand and shrink the window to find the maximum length.

Pseudocode:

```
void lengthOfLongestSubstring(string s) {  
    unordered_set charSet;  
    int left = 0, maxLength = 0;  
    for (int right = 0; right < s.size(); right++) {  
        while (charSet.count(s[right]))
```



```

{
charSet.erase(s[left]);
left++;
}
charSet.insert(s[right]);
maxLength = max(maxLength, right - left + 1);
}
cout << maxLength;
}

```

Dry Run:

Input: "abcabcbb"

Window: [a, b, c] → Length = 3.

Shrink and expand → Length = 3. Output: 3

Time Complexity: $O(N)$.

Space Complexity: $O(N)$.

10. Meeting Rooms

Approach 1: Sorting by End Time

- Sort meetings by their ending times.
- Iterate and check if the next meeting starts after the previous one ends.

Pseudocode:

```

void meetingRooms(vector<pair<int, int>> &meetings) {
    sort(meetings.begin(), meetings.end(), [](pair &a, pair &b)

```